

Μεταγλωττιστές 2025-26

Συντακτική ανάλυση LL(1):
Σύνολα FIRST

Περιορισμοί γραμματικών που μπορούμε να χειριστούμε ως τώρα

- Θα πρέπει όλα τα δεξιά μέρη των κανόνων να ξεκινούν με **τερματικό σύμβολο**
 - Φυσικά, για να είναι LL(1) η γραμματική θα πρέπει **τα τερματικά σύμβολα** που ξεκινούν το δεξιό μέρος κάθε εναλλακτικού κανόνα **του ίδιου μη τερματικού συμβόλου** να είναι **διαφορετικά μεταξύ τους**
- Θα πρέπει να μην εμφανίζονται **κενές παραγωγές** (ϵ)
- Το παράδειγμα που είδαμε ως τώρα:

$S \rightarrow a B$

$B \rightarrow b \mid a B b$

Παράδειγμα σε στυλ “Logo graphics”

```
CommandList → [ Command CommandTail
CommandTail → , Command CommandTail | ]
Command      → forward Intvalue | backward Intvalue
               | left Intvalue | right Intvalue
               | up | down
               | repeat num CommandList
Intvalue      → + num | - num | num
```

```
[ forward 50, backward -20, left +60, repeat 4 [ right 70 ], forward +120 ]
```

- Οι περιορισμοί στη μορφή της γραμματικής οδηγούν σε μια «μη κομψή» εκδοχή της γλώσσας
 - Περισσότερα κόμματα, τετράγωνες αγκύλες...
 - Μπορούν να αποφευχθούν με μια πολύ πιο «άσχημη» γραμματική

Χαλάρωση των περιορισμών

- ~~Θα πρέπει όλα τα δεξιά μέρη των κανόνων να ξεκινούν με **τερματικό σύμβολο**~~
 - Μπορούμε να βρούμε έναν τρόπο ώστε τα δεξιά μέρη των κανόνων να αρχίζουν **και με μη τερματικό σύμβολο**;
 - Η γραμματική θα πρέπει να παραμείνει LL(1)
 - Αν το δεξιό μέρος ενός κανόνα ξεκινάει με μη τερματικό, θα πρέπει **να μην υπάρχει αριστερή αναδρομή** (έστω και έμμεσα)
 - Όταν υπάρχουν εναλλακτικοί κανόνες παραγωγής **για το ίδιο μη τερματικό σύμβολο** (αριστερό μέρος κανόνα), θα πρέπει να μπορούμε να διαλέξουμε **μονοσήμαντα** κανόνα ανάλογα με το token που βρίσκεται τώρα στην είσοδο

Παράδειγμα

Κανόνες

```
Session -> Fact Session  
         | Question  
         | ( Session ) Session
```

```
Fact -> ! string
```

```
Question -> ? string
```

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

```
((?query0) !input1 !input2 ! input3 ?query1 ) ? query2
```

Διατύπωση του προβλήματος

- Αν το δεξιό μέρος ενός κανόνα **ξεκινά με τερματικό σύμβολο**, γνωρίζουμε πώς θα τον επιλέξουμε για υλοποίηση:

```
def Fact():  
    if next_token=='!':  
        # Fact -> ! string  
        match('!')  
        match(string)
```

- Αν **ξεκινά με μη τερματικό σύμβολο**, πώς γίνεται η επιλογή;

```
def Session():  
    if next_token==?????:  
        # Session -> Fact Session  
        Fact()  
        Session()
```

- Αν ξέραμε **όλα τα πιθανά τερματικά σύμβολα** με τα οποία ξεκινούν οι προτάσεις που παράγει το **Fact**;

Σύνολα FIRST

- **FIRST(x)** είναι το σύνολο των τερματικών συμβόλων από τα οποία ξεκινά κάθε πρόταση που παράγεται από το **x** σε μηδέν ή περισσότερα βήματα
 - Όπου **x** οποιαδήποτε ακολουθία τερματικών και μη τερματικών συμβόλων
- Αν **x** ξεκινά με **τερματικό σύμβολο a**, το FIRST(x) περιέχει το ίδιο το **a**
- Αν **x** ξεκινά με **μη τερματικό σύμβολο A**, το FIRST(x) ισούται με το **FIRST(A)**
 - σημ: εάν το **A** παράγει **ε** (κενή παραγωγή), το FIRST(x) μπορεί να περιλαμβάνει και άλλα στοιχεία
 - θα το δούμε αργότερα

Εύρεση συνόλων FIRST

(όταν δεν υπάρχουν κενές παραγωγές με ϵ)

- Ξεκινάμε με **άδεια** σύνολα FIRST για κάθε μη τερματικό σύμβολο της γραμματικής
- Για κάθε κανόνα της γραμματικής:
 - Αν το δεξιό μέρος του κανόνα ξεκινά με **τερματικό a** , προσθέτουμε το a στο σύνολο FIRST του μη τερματικού που βρίσκεται στο αριστερό μέρος του κανόνα
 - Αν το δεξιό μέρος του κανόνα ξεκινά με **μη τερματικό A** , προσθέτουμε τα σύμβολα του FIRST(A) που ξέρουμε εκείνη τη στιγμή στο σύνολο FIRST του μη τερματικού που βρίσκεται στο αριστερό μέρος του κανόνα
- Επαναλαμβάνουμε τη διαδικασία από την αρχή, έως ότου να μην μπορεί να προστεθεί άλλο σύμβολο σε κάποιο σύνολο FIRST
 - Ο αλγόριθμος τερματίζει εγγυημένα: ο αριθμός των επαναλήψεων που θα γίνουν φράσσεται από τον αριθμό των τερματικών και μη τερματικών συμβόλων, που είναι πεπερασμένος.

Παράδειγμα

Σύνολα FIRST	Κανόνες
(	<code>Session -> Fact Session Question (Session) Session</code>
!	<code>Fact -> ! string</code>
?	<code>Question -> ? string</code>

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Παράδειγμα

Σύνολα FIRST	Κανόνες
!	Session -> Fact Session
?	Question
(	(Session) Session
!	Fact -> ! string
?	Question -> ? string

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Χρήση συνόλων FIRST (γραμματικές χωρίς ε)

- Επιλέγουμε την υλοποίηση ενός κανόνα $A \rightarrow x$ για κάθε token που ανήκει στο **FIRST(x)**
 - Όταν το FIRST(x) περιέχει πολλά tokens, τα βάζουμε όλα στη συνθήκη του if (με or)

```
def Session():  
    if next_token=='!':  
        # Session -> Fact Session  
        # FIRST(Fact Session) = FIRST(Fact) = { ! }  
        Fact()  
        Session()  
    elif ...
```

- Προσοχή: για κάθε εναλλακτικό κανόνα του ίδιου μη τερματικού συμβόλου θα πρέπει τα σύνολα FIRST των δεξιών μερών να μην έχουν κοινά στοιχεία αλλιώς η γραμματική δεν είναι LL(1)!

“Logo graphics” – Βελτιωμένη γραμματική

CommandList → [Command CommandTail

CommandTail → Command CommandTail |]

Command → Move | Turn | Pen | repeat num CommandList

Move → forward Intvalue | backward Intvalue

Turn → left Intvalue | right Intvalue

Pen → up | down

Intvalue → + num | - num | num

```
[ forward 50  
  backward -20  
  left +60
```

```
repeat 4 [  
  right 70  
]
```

```
forward +120  
]
```

- Μπορούμε να αποφύγουμε περιττά στοιχεία και να έχουμε μια πιο «κομψή» γραμματική
- Απέχει ακόμα από το ιδανικό λόγω της μη υποστήριξης κενών παραγωγών (ε)